Project 2

Due Date: Monday 8 July 2024 by 8:00 AM

General Guidelines.

The instructions given below describe the required methods in each class. You may need to write additional methods or classes that will not be directly tested but may be necessary to complete the assignment.

In general, you are not allowed to import or use any additional classes in your code without explicit permission from your instructor!

Note: It is okay to use the Math class if you want.

Project Overview

In this project, you will use a double-ended queue (aka a deque), which can be used as both a Stack and a Queue in order to solve word search puzzles. You will need to do this by implementing versions of breadth-first search (BFS) and depth-first search (DFS) in a grid of letters.

Note: Only properly submitted projects are graded! No projects will be accepted by email or in any other way except as described in this handout.

Word Search

In this part, you will use a variation of DFS and BFS with a Deque structure to solve a word search. In this part, the grid you are given will contain individual letters represented as Strings, and you must implement a word search. A word can occur as any path from the first letter to the last where a step in the path can go up, right, down, or left. (No diagonals). Also, it is possible for the same letter to be used more than once in the same word. (See ARIZONA example below.)

For help understanding BFS and DFS, see section 14.3 in the textbook. Also, feel free to come to office hours or SI, and we can go through them with you as well. Note that this project requires some variation on these algorithms, but the basics are still DFS and BFS.

	0	1	2	3	4	5	6	7	8	9
0	A	N	O	D	E	F	G	H	I	J
1	R	O	N	N	O	P	Q	W	E	R
2	I	Z	A	E	R	P	A	S	D	F
3	G	H	J	V	S	L	G	H	J	K
4	A	S	W	S	R	F	F	G	H	J
5	B	A	M	N	I	V	C	X	Z	A
6	S	W	Y	U	R	E	V	T	B	Y
7	N	D	C	I	S	R	P	O	G	H
8	I	L	A	T	Y	S	C	V	R	F
9	W	S	T	A	S	D	F	G	H	E

The output for this search should be a String representation of the locations in the path as shown below.

Examples:

```
ARIZONA: (0, 0)(1, 0)(2, 0)(2, 1)(1, 1)(0, 1)(0, 0)
UNIVERSITY: (6, 3)(5, 3)(5, 4)(5, 5)(6, 5)(7, 5)(7, 4)(7, 3)(8, 3)(8, 4)
WILDCATS: (9, 0)(8, 0)(8, 1)(7, 1)(7, 2)(8, 2)(9, 2)(9, 1)
```

Additionally, the search function will take two integer parameters that indicate the row and column that your search will start from.

Your implementation should follow these guidelines.

- Use a BFS algorithm to find the first character in the word. This search will not be included in the final path that is printed out, but it is included in the overall grid access counts, so it is important to use BFS in order to find the closest option first and go on from there.
- Once you find a possible starting place for the word, use a DFS algorithm to search for the word itself.

- **Always search a location's neighbors in the following order: UP, RIGHT, DOWN, LEFT. If you don't, you may get an alternate route that does not match the test cases.**

Example: Let's say we are searching the above grid for ARIZONA starting at (2, 3). First we search for the closest "A" using a BFS algorithm, which is at (2, 2). We then use DFS to search for the word. When we don't find it, we continue the BFS for the next closest "A". Eventually, we find the correct "A" and find the path as noted above.

Testing.

- Test cases and test code is provided for you. Keep in mind that these may not necessarily catch all possible errors, so it's up to you to make sure you are handling any corner cases.
- Note: The test cases search for multiple words in the same grid, so you will need to reset some things either at the beginning or end of each word search in order to make sure you get accurate results in the subsequent searches. You are allowed to use a single ArrayList to keep track of the items to be reset.
- To avoid any differences in String output, use the *toString* method provided in *Loc.java* to print out the locations. Do not add any spaces or characters in between them.

Provided Code.

Besides the test code, you are provided with the *Grid.java* class. You should NOT change this code as it will not be included in your submission. However, you may change the *Loc.java* class, and you should include your version of *Loc.java* in your submission even if you don't change it. (But you probably should change it.) You are also provided with the *Deque.java* and the *EmptyDequeException.java* class, which you should not change. You are not allowed to use other structures (except for the ArrayList mentioned above), but a Deque should be enough for your BFS and DFS implementations as it can function as both a Stack and a Queue.

API.
Implement your solution in a class called *Puzzle.java*. The required methods are listed below.

Method Signature	Description
`public Puzzle(Grid grid)`	`constructor`
`public String find(String word, int r, int c)`	`find and return (as a String) the path containing word; start the search at (r, c)`

REMINDER: Always search a location's neighbors in the following order: UP, RIGHT, DOWN, LEFT. If you don't, you may get an alternate route that does not match the test cases.

Note on Efficiency Testing. Unfortunately, there is no foolproof way to autograde for efficiency. The tests provided are meant to give you a reasonable idea of whether your solutions fits within the expected efficiency level and what is possible. Ultimately, we can always check your code manually if necessary (we do some of this already) to verify that the autograder is returning reasonable results. Please note, however, that the access counts need to be accurate according to your solution, so don't do something that doesn't actually improve your solution just to get the access counts down. (That's a waste of your time and could be considered academic dishonesty.)

Submitting, Testing, and Grading

Your code must compile and run on lectura, and it is up to you to check this before submitting.
To test your code on lectura:

- Transfer all the files (including test files) to lectura.
- Run *javac *.java* to compile all the java files. (You can also use *javac* with an individual java file to compile just the one file: *javac ArrList.java*)
- Run *java <filename>* to run a specific java file. (Example: *java ArrListTest.java*)

After you are confident that your code compiles and runs properly on lectura, you can submit the required files using the following **turnin** command. Please do not submit any other files than the ones that are listed.

```
turnin csc345pa2 Puzzle.java Loc.java
```

Upon successful submission, you will see this message:

Turning in:

```
Puzzle.java -- ok
Loc.java -- ok
```

All done.

Note: Only properly submitted projects are graded! No projects will be accepted by email or in any other way except as described in this handout and the syllabus. If you are worried about your submission, feel free to ask us to check that it got into the right folder.

Grading.

Your score will be determined by the tests we do on your code (which are similar to the provided tests) minus any deductions that are applied when your code is manually graded. In addition to late deductions, coding style, and deductions for not following directions, you may also receive deductions for inefficient code.

For Coding Style, we look for:

- understandable code
- helpful comments
- good indentation
- good method abstraction–i.e. you should be writing reusable methods rather than repeating the same code multiple times

See the resubmission policy in the syllabus.

It is absolutely not okay to ask ChatGPT to give you a solution to this problem, nor is it okay for you to get solutions from any other sources or to share your solution with anyone else.